DIGITAL EGYPT PIONEERS INITIATIVE • DEPI

Data Engineering Track • Graduation Project

Project Documentation

Part 1 • Planning, Literature Review & Requirements

Web-to-Warehouse Product Intelligence Pipeline

An automated, end-to-end data engineering pipeline that turns public web product pages into a clean, historical, analysis-ready warehouse, and tells you what changed.

PROJECT TEAM

Marwa Yosry مروة يسري Team Leader | Member 1	**Gany Mohamed** جني محمد Member 2
Ayat Abdelaziz آيات عبدالعزيز Member 3	**Carol Kitchener** كارول كيتشنر Member 4
Engy Ashraf إنجي أشرف Member 5	**Track** Data Engineering (DEPI) Graduation Project, Phase 5

DOCUMENT CONTENTS

1. Project Planning & Management | 2. Literature Review | 3. Requirements Gathering

GitHub: https://github.com/marwa698/product-intelligence-pipeline

October 2026

Table of Contents

1. Project Planning & Management . . . 3

1.1 Project Proposal . . . 3

1.2 Project Plan . . . 7

1.3 Task Assignment and Roles . . . 8

1.4 Risk Assessment and Mitigation Plan . . . 9

1.5 Key Performance Indicators (KPIs) . . . 9

2. Literature Review . . . 11

2.1 Existing Solutions . . . 11

2.2 Comparison with Our Project . . . 11

2.3 Techniques Reviewed . . . 12

2.4 Gaps Found and How We Address Them . . . 13

2.5 Feedback, Evaluation and Suggested Improvements . . . 13

2.6 References . . . 13

3. Requirements Gathering . . . 14

3.1 Stakeholder Analysis . . . 14

3.2 User Stories and Use Cases . . . 14

3.3 Functional Requirements . . . 16

3.4 Non-Functional Requirements . . . 17

3.5 Assumptions and Constraints . . . 18

3.6 Traceability: Requirements to Team Members . . . 18

About this document

This document is **Part 1** of the Graduation Project documentation required by DEPI: *Project Planning & Management*, *Literature Review* and *Requirements Gathering*. Parts 4 to 7 (System Analysis & Design, Implementation, Testing and Final Presentation) will follow on the dates set in the Project Documentation Guidelines. All project files are in the team GitHub repository.

1. Project Planning & Management

1.1 Project Proposal

1.1.1 Overview

The **Web-to-Warehouse Product Intelligence Pipeline** is an automated data engineering system that continuously collects product information (name, category, price, rating, availability and product URL) from a permitted public website, validates and standardises it, detects duplicate products, compares it with an internal product catalogue, and loads it into a MySQL analytical database. Every run is stored as a **dated snapshot**, so analysts can answer the question that raw scraping cannot: *what has changed since the last run?* (price changes, new products, removed products and category changes). The whole workflow is orchestrated by Apache Airflow, packaged with Docker and presented through a dashboard. On top of the core pipeline, the team adds four production-grade enhancements: **failure and price alerts** (Telegram/email), a **Pipeline Health page** backed by a run-log table, **automated tests with GitHub Actions CI**, and a documented **REST API** (FastAPI + Swagger).

1.1.2 Problem Statement

A retailer wants to keep track of product information published on public web sources and combine it with its own product catalogue. Today this is done manually, which is slow, produces inconsistent records and keeps no history. Collected data typically contains duplicated products, mixed price formats (for example **£51.77** as text), text ratings such as *Three*, missing values, and no way to tell whether a product was added, removed, re-priced or moved to another category between two collections.

1.1.3 Project Objectives

- **Collect** all product records (1,000 in the chosen source) automatically, politely and reliably, with retries and logging.
- **Standardise** and validate the data: numeric prices with a currency code, integer ratings (1 to 5), consistent availability and category names, with rejected rows kept separately.
- **Detect duplicates** using exact matching on product URL and fuzzy matching on product names, and compare records with an internal catalogue.
- **Store** the data in a relational model (staging, products, price history) that preserves a snapshot per product per day.
- **Analyse** with SQL views for price changes, new products, removed products and category changes.
- **Automate** the full flow with an Airflow DAG (schedule, retries, failure callback) and make the environment reproducible with Docker Compose.
- **Guarantee quality**: run data-quality checks on every load and make the pipeline *idempotent* (re-running never creates duplicates).
- **Present** the results in an interactive dashboard and through a documented REST API (FastAPI with Swagger).
- **Operate like production**: send alerts on pipeline failures and large price movements, show pipeline health (duration, rows loaded and rejected), and run automated tests on every pull request (GitHub Actions CI).

1.1.4 Scope

In scope	Out of scope (for this project)
Scraping a permitted public product website (books.toscrape.com) with BeautifulSoup/Requests	Scraping websites that forbid it or require a login
Cleaning, validation, duplicate detection with Pandas and RapidFuzz	Collecting personal or customer data
MySQL schema, ERD, indexes, SQL views	Automatic repricing / dynamic pricing decisions
Airflow DAG, Docker Compose environment	Cloud deployment and paid infrastructure
Data-quality checks, rejected-row handling, quality log	Real-time (streaming) ingestion
Dashboard (Streamlit + Plotly) and a REST API (FastAPI + Swagger)	Mobile application
Alerts (Telegram or email) and a Pipeline Health page with a run-log table	
Automated tests (pytest) and GitHub Actions CI	
Documentation, README, architecture diagrams	

1.1.5 Proposed Solution at a Glance

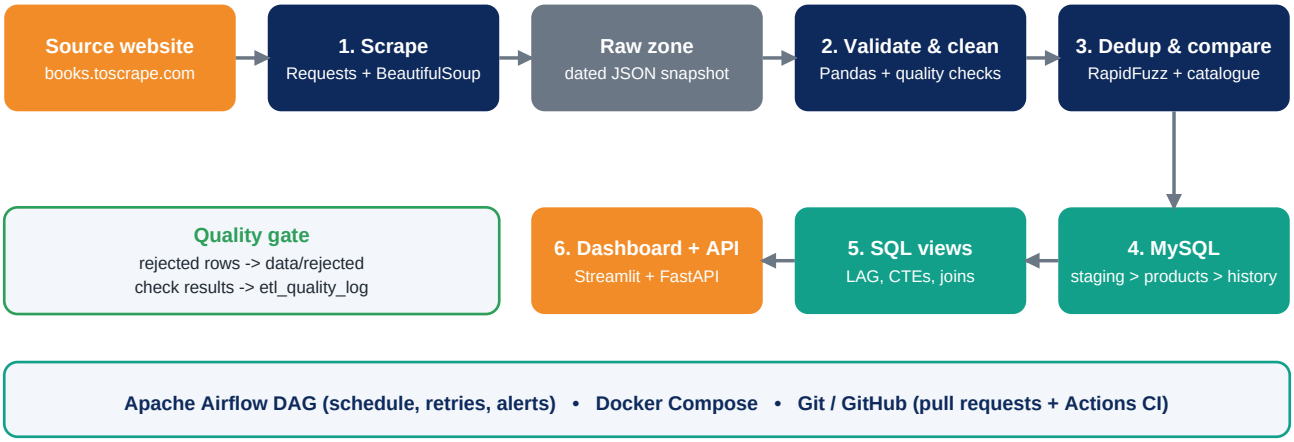


Figure 1. High-level flow of the pipeline. Airflow runs the steps in order every day; failed tasks are retried.

1.1.6 Data Source and Ethical Scraping

The data source is **books.toscrape.com**, a sandbox website created specifically for practising web scraping. It lists 1,000 books across 50 pages and 50 categories, each with a name, category, price, rating, availability and URL. Before scraping, the team checked */robots.txt* (the server returned **404 Not Found**, so no crawling restrictions are declared; the Robots Exclusion Protocol is standardised in RFC 9309 [5]) and recorded the result in the repository. The scraper follows good-citizen rules: a **1-second delay** between requests, an identifiable User-Agent, no login and no private pages.

> **Important note about the data**
>
> The website itself states that prices and ratings are **randomly assigned** and have no real meaning. The project therefore does not depend on real market prices: it demonstrates the *engineering* (collection, cleaning, modelling, change detection, orchestration). Price-change detection is verified with repeated daily runs and a controlled second snapshot.

1.1.7 Why This Project Stands Out

Many student projects stop at "a script that scrapes a website". This project delivers the **complete data engineering lifecycle** and treats the data like a production asset:

#	What makes it different	Why it matters / evidence
1	**Complete lifecycle in one system**: ingest, validate, clean, deduplicate, model, orchestrate, monitor, visualise.	Covers every core skill of the DEPI Data Engineering track (Python, Pandas, SQL, MySQL, Airflow, Docker, Git) in one coherent product.
2	**History, not a copy**: one dated snapshot per product per day (periodic-snapshot design).	Turns static data into *insight*: price movements, new/removed products and category changes are found with SQL window functions.
3	**Two-layer duplicate detection**: exact (URL) plus fuzzy name matching, with human review.	Fuzzy matches are reported, not silently deleted, which protects against wrongly merging different products.
4	**Idempotent loading**: staging table, upsert and a unique (product, day) rule inside one transaction.	Re-running the pipeline never duplicates data. This is a production requirement that is verified by a dedicated test.
5	**Quality gate and rejected-row handling**: checks on nulls, ranges, duplicates and row counts, logged to a table.	Bad data is isolated and visible; a failed check fails the Airflow task instead of passing silently.
6	**Proven resilience on real data**: the first full run collected 951 of 1,000 products; all 49 failures were network errors. A retry mode recovered them, giving 1,000 of 1,000 unique records.	Shows the pipeline was tested against real failures (DNS and timeout problems) and recovers without manual cleanup.
7	**Ethical and legal by design**: robots.txt check, rate limiting, identifiable agent, no personal data.	Responsible data collection, documented in the repository.
8	**Professional team workflow**: five members, a written Data Contract, feature branches, pull requests into a protected main branch, secrets kept out of Git.	Demonstrates real software-engineering collaboration and a defined hand-off interface between components.
9	**Reproducible**: the environment starts with one Docker Compose command.	Anyone (instructor, evaluator, company) can run it without manual setup.
10	**Operational visibility**: Telegram/email alerts on failures and large price moves, plus a Pipeline Health page (run durations, rows loaded and rejected).	Problems are noticed by the team first, not by business users. This addresses the observability gap that real pipelines suffer from.
11	**Quality engineering and open access**: pytest tests run automatically on every pull request (GitHub Actions CI), and the data is served through a documented REST API (Swagger).	A broken change cannot reach main unnoticed, and other applications can consume the data without touching the database.

1.1.8 Expected Deliverables

Deliverable (from the project brief)	Owner(s)	Status
Scraper / API ingestion module	Member 1	Done (merged to main)
Database schema, ERD and historical price table	Member 2	In progress
ETL pipeline: cleaning, duplicate detection, loading	Members 3 and 4	In progress
Data-quality checks and quality log	Member 3	Planned
SQL analysis (views for changes, new, removed, category changes)	Member 2	Planned
Airflow DAG and Docker Compose environment	Member 5 with Team Leader	Planned
Visualisation (dashboard)	Member 1	Planned
Enhancement: REST API (FastAPI + Swagger)	Member 4	Planned
Enhancement: alerts (Telegram / email)	Member 5 with Team Leader	Planned

Deliverable (from the project brief)	Owner(s)	Status
Enhancement: Pipeline Health page and run-log table	Members 1, 2 and 5	Planned
Enhancement: automated tests and GitHub Actions CI	Members 3 and 5	Planned
Documentation: README, architecture diagram, screenshots of Airflow runs	Member 5 and Team Leader	Planned

1.1.9 Team-Selected Enhancements

After the core pipeline, the team committed to four enhancements. They are scheduled after the core milestones so that they can never put the required deliverables at risk.

Enhancement	What it delivers	Value	Owner(s)
1. Alerts	A message on Telegram (or email) when a DAG task fails after its retries, and when a product price moves by more than a configurable threshold (default 20%).	Faster reaction; shows monitoring maturity.	Member 5 + Team Leader
2. Pipeline Health page	A *pipeline_run_log* table (start, end, duration, status, rows scraped, loaded, rejected) and a dashboard page showing the latest runs and trends.	Makes pipeline behaviour visible to non-engineers.	Members 2, 5 and 1
3. CI with GitHub Actions	pytest unit tests (cleaning, deduplication) and an idempotency check run automatically on every pull request.	Protects the main branch; satisfies the CI/CD item of the guidelines.	Members 3 and 5
4. REST API	FastAPI endpoints */products/latest*, */products/price-changes*, */products/new*, */products/removed* with automatic Swagger documentation.	Lets other applications use the data; satisfies the API documentation item.	Member 4

1.2 Project Plan

1.2.1 Approach

The team works in short weekly iterations. Work is split into independent components (scraper, database, cleaning, loading, orchestration, dashboard) that connect through a written **Data Contract** (agreed column names and types) and agreed function names. Each member works on a feature branch and merges through a pull request, so the main branch always contains working code. Members start in parallel on small sample data and integrate during the testing phase.

1.2.2 Timeline (Gantt Chart)



Figure 2. Project timeline (weeks W1 to W8). Green = completed, blue = documentation, teal = implementation, orange = testing and delivery. Diamonds mark the DEPI document deadlines.

1.2.3 Milestones and Deliverables

Milestone	Date	Deliverable / exit criterion
M1 Project Planning, Literature Review, Requirements	19 Oct 2026	Part 1 PDF uploaded to GitHub and submitted on the LMS
M2 Raw data collected	Completed	1,000 unique products in JSON, scraper merged to main
M3 System Analysis and Design	6 Nov 2026	ERD, schemas, DFD, diagrams, UI mock-ups, technology stack
M4 Core pipeline loads MySQL	16 Nov 2026 (team target)	Clean, deduplicate, load; idempotency test passed
M5 Orchestration, dashboard and enhancements	27 Nov 2026 (team target)	Airflow DAG green end to end; dashboard with 3-4 charts; API, alerts, health page and CI working
M6 Implementation	30 Nov 2026	Source code, README, Docker environment, execution guide
M7 Final presentation, testing and reports	4 Dec 2026	Test report, user manual, technical documentation, slides, infographic

1.2.4 Resource Allocation

Resource	Details
People	Five team members with clearly separated components (see section 1.3); one Team Leader.
Software and libraries	Python 3.10; Requests, BeautifulSoup, lxml, Tenacity; Pandas, RapidFuzz; SQLAlchemy, PyMySQL; MySQL 8 (with MySQL Workbench); Apache Airflow; Docker and Docker Compose; Streamlit, Plotly; FastAPI and Uvicorn (REST API); pytest; GitHub Actions; Telegram Bot API or SMTP (alerts).
Tools	Git and GitHub (private/public repository, pull requests), VS Code, dbdiagram.io or draw.io, a WhatsApp group for daily coordination.
Infrastructure	Team laptops running Docker Desktop; GitHub for hosting. No paid services are required.
Data	books.toscrape.com (1,000 products) and a small internal catalogue seeded for comparison testing.
Time	About 8 weeks (12 Oct to 4 Dec 2026), aligned with the DEPI documentation deadlines.

1.3 Task Assignment and Roles

#	Team member	Responsibilities	Main deliverables
1	**Marwa Yosry** مروة يسري Team Leader	Web scraper (pagination, rate limiting, retry, logging); robots.txt compliance; raw dataset; dashboard and visualisations; Pipeline Health page; repository governance and the Data Contract.	scrape_products.py, raw JSON, dashboard, health page, robots note
2	**Gany Mohamed** جني محمد Database and SQL	MySQL schema, ERD, indexes; internal catalogue; SQL analysis (LAG, CTEs, joins) and views for price changes, new, removed and category-changed products; pipeline_run_log table for the health page.	schema.sql, views.sql, ERD, analysis queries, run-log table
3	**Ayat Abdelaziz** آيات عبدالعزيز Cleaning and data quality	Schema validation; cleaning of names, price, currency, rating, availability; missing-value rules; rejected rows; data-quality checks and quality log; pytest unit tests for cleaning.	clean.py, quality_checks.py, rejected files, quality log, tests
4	**Carol Kitchener** كارول كيتشنر Deduplication and loading	Exact and fuzzy duplicate detection (RapidFuzz); comparison with the internal catalogue; staging, upsert and history loading in one transaction; database connection module; REST API (FastAPI + Swagger).	dedup.py, load.py, db.py, run_etl.py, API
5	**Engy Ashraf** إنجي أشرف Orchestration and integration (with the Team Leader)	Docker Compose environment; Airflow DAG (schedule, retries, failure callback, logging); integration and end-to-end testing; alerts (Telegram/email); GitHub Actions CI workflow; run-log writing; README and architecture documentation; Airflow run screenshots.	docker-compose.yml, product_pipeline.py, alerts, CI workflow, README, screenshots

Collaboration rules

- Every member works on their own **feature branch**; nobody pushes directly to *main*. Changes enter through a **pull request**, reviewed by the Team Leader.
- Interfaces between components are fixed in a **Handoff Interface** (function names, inputs and outputs) and the **Data Contract** (column names and types).
- Secrets (passwords) live in a *.env* file that is excluded from Git; only *.env.example* is committed.
- Day 1 hand-offs unblock everyone: a 10-product sample file (Member 1), the SQL schema (Member 2), the database connection module (Member 4).
- Progress is tracked in the team group and on GitHub (branches, pull requests, commit history).

1.4 Risk Assessment and Mitigation Plan

Likelihood (L) and impact (I) are rated Low / Medium / High. The first two risks have already occurred during development.

#	Risk	L	I	Mitigation
R1	**Unstable internet / DNS failures** during scraping (occurred: 49 of 1,000 requests failed in the first run).	High	Med	3 automatic retries per request; failures logged; a retry-failed mode re-collects only missing products; Airflow task retries.
R2	**Components do not fit together** because members use different column or function names.	Med	High	Written Data Contract and Handoff Interface; sample data on day 1; integration phase with end-to-end tests.
R3	Source website changes its HTML structure or blocks requests.	Low	High	Selectors isolated in one module; polite rate limit; validation fails loudly on empty fields; raw data kept for replay.
R4	Demo website prices are random, so change detection may show unrealistic movement.	Med	Low	Documented in the proposal; verify logic with a controlled second snapshot and repeated runs.
R5	Duplicate or dirty data corrupts the warehouse.	Med	High	Staging table, unique constraints, upsert in a transaction, quality checks, rejected-row files.
R6	Merge conflicts or broken main branch.	Med	Med	Feature branches, pull requests, protected main branch, small frequent commits.
R7	Docker / Airflow environment problems on team laptops (resources, virtualisation).	Med	Med	Shared docker-compose file; documented setup; fall back to running Python modules directly for development.
R8	Credentials accidentally committed to a public repository.	Low	High	.gitignore for .env and logs; .env.example only; review of git status before every commit.
R9	Time pressure or member unavailability close to deadlines.	Med	High	Parallel work from day 1; clear ownership; Team Leader reallocates tasks; optional features (API) are cut first.
R10	Scope creep from extra features.	Med	Med	Must-have scope fixed in section 3; the four enhancements start only after the core pipeline passes testing, in this order: tests/CI, alerts, health page, API.
R11	Alert credentials (Telegram bot token, email password) leak to the public repository.	Low	High	Tokens only in .env and GitHub Secrets; .env ignored by Git; tokens rotated if exposed.

1.5 Key Performance Indicators (KPIs)

KPI	Target	How it is measured	Current status
Scraping completeness	1,000 of 1,000 products	Count of unique product URLs in the raw JSON	**Achieved**: 1,000 unique (951 on first pass, 49 recovered by retry)
Scraper success rate (first pass)	>= 99%	Successful requests / total requests, from scraper.log	95.1% (network failures only); fixed by retry mode
Pipeline run success rate	>= 95% of scheduled runs green	Airflow DAG run history	To be measured
Idempotency	0 extra rows after a re-run	Row counts in products and price_history before and after a second run	To be tested

KPI	Target	How it is measured	Current status
Data validity	>= 95% rows pass cleaning; rejected < 5%	Quality-check results in etl_quality_log	To be measured
Duplicates in warehouse	0	Unique constraint on product_url; duplicate check	To be tested
Processing time (clean, dedup, load)	< 2 minutes for 1,000 records	Airflow task durations	To be measured
Analytical query response	< 2 seconds per view	EXPLAIN ANALYZE / timing on views	To be measured
Change-detection correctness	100% of seeded changes found	Test snapshot with known price, category, new and removed items	To be tested
Automation	All 7 DAG tasks green end to end	Airflow graph view and screenshots	To be demonstrated
Alert timeliness	Failure alert within 2 minutes after the last retry fails	Forced failure test in Airflow	To be tested
CI health	100% of pull requests to main pass the test workflow	GitHub Actions run history	To be measured
API response time	< 500 ms for /products/latest (50 items)	Swagger / timing test	To be measured
Pipeline Health coverage	Last 30 runs visible with duration, rows loaded and rejected	Dashboard page and run-log table	To be demonstrated
Team delivery	All pull requests merged before each milestone	GitHub pull requests and milestones	On track (Part 1)

2. Literature Review

This section reviews existing solutions and techniques related to price and product monitoring, identifies their strengths and limitations, and explains how our project differs and what it borrows from them. Sources are listed in section 2.6.

2.1 Existing Solutions

Solution	What it does	Strengths	Limitations (relative to our goals)
Keepa [1] (Amazon price tracker)	Tracks Amazon product prices over time and shows price-history charts, alerts, a browser extension and an API.	Very large product coverage; clear price-history charts; alerts when a target price is reached.	Focused on one marketplace for shoppers and sellers; a closed service, so users cannot control cleaning, duplicate logic or compare with their own internal catalogue.
CamelCamelCamel [2]	Amazon price tracker with price-history charts and price-drop alerts.	Free, simple, easy for consumers.	Consumer tool for one marketplace; no warehouse, no custom data model, no orchestration or quality framework.
Prisync [3] (commercial SaaS)	Competitor price tracking and dynamic-pricing software for e-commerce, with price history, notifications and integrations.	Mature product aimed at retailers; dashboards, alerts and repricing rules.	Paid subscription; internal pipeline is a black box, so it teaches or proves nothing about the engineering (deduplication, quality gates, orchestration) and cannot be adapted freely.
Scraping frameworks and no-code scrapers [4][6] (Scrapy, BeautifulSoup, visual scrapers)	Extract data from web pages into files or databases.	Flexible and powerful for extraction; large communities.	Extraction only: no historical modelling, no duplicate detection, no scheduling or quality checks unless built around them.
Open-source hobby price trackers [7] (GitHub "price-tracker" projects)	Typically a script that checks a URL and notifies by email or writes to a file.	Easy to start; useful as inspiration.	Usually a single script for a few products, with little staging, validation, idempotency, orchestration or tests.

2.2 Comparison with Our Project

Capability	Keepa / CamelCamelCamel	Prisync	Scrapers only	Hobby scripts	Our project
Collects many products automatically	Yes	Yes	Yes	Partial	**Yes**
Keeps full price history per product	Yes	Yes	No	Partial	**Yes**
Detects new / removed / category-changed products	Partial	Partial	No	No	**Yes (SQL views)**
Duplicate detection (exact + fuzzy)	Not exposed	Not exposed	No	No	**Yes**
Compares with an internal catalogue	No	Partial	No	No	**Yes**
Cleaning, validation, rejected-row handling	Not exposed	Not exposed	No	No	**Yes**

Capability	Keepa / CamelCamelCamel	Prisync	Scrapers only	Hobby scripts	Our project
Automated data-quality checks and log	Not exposed	Not exposed	No	No	**Yes**
Scheduled orchestration with retries	Internal	Internal	No	No	**Yes (Airflow)**
Pipeline-failure alerts and health page	Not exposed	Not exposed	No	No	**Yes**
Automated tests and CI on every change	Not exposed	Not exposed	No	No	**Yes**
Open, adaptable data model	No	No	Yes	Yes	**Yes**
Reproducible with one command (Docker)	No	No	Partial	No	**Yes**
Cost	Free / paid tiers	Paid	Free	Free	**Free**

Table notes: "Not exposed" means the internal behaviour is not visible or configurable by the user. The comparison reflects the publicly described purpose of each solution at the time of writing and is intended as a high-level positioning, not a feature audit.

2.3 Techniques Reviewed

Responsible web scraping

The Robots Exclusion Protocol (RFC 9309) [5] lets site owners declare which paths crawlers may visit. Good practice also includes rate limiting and an identifiable User-Agent. **Adopted:** robots.txt check, 1-second delay, identifiable agent, retry with back-off.

ETL, staging and idempotent loading

Loading raw data into a temporary staging area and then merging (upsert) into target tables is a standard data-warehouse pattern; it isolates bad data and makes loads repeatable. **Adopted:** *stg_products* staging table, upsert into *products*, unique (product, day) rule in *price_history*, all in a single transaction.

Dimensional modelling and history

Kimball and Ross [8] describe periodic-snapshot fact tables and slowly changing dimensions for keeping history. **Adopted:** a periodic snapshot of price, rating, availability and category per product per day, plus *first_seen*, *last_seen* and *is_active* on the product dimension, which allows change detection with SQL window functions.

Entity resolution and fuzzy matching

Edit-distance measures such as Levenshtein distance [9] quantify how similar two strings are; the RapidFuzz library [10] provides fast implementations. **Adopted:** a similarity threshold of about 90% to *flag* likely duplicates for review rather than delete them automatically.

Data quality

Data quality is commonly described by dimensions such as completeness, validity, uniqueness and consistency. Frameworks such as Great Expectations [11] automate such checks. **Adopted:** a lightweight, custom set of checks (null rate, ranges, duplicates, row-count ratio, rejected ratio) whose results are stored in a log table and which fail the pipeline when violated.

Workflow orchestration and containers

Apache Airflow [12] represents workflows as directed acyclic graphs (DAGs) with scheduling, retries and monitoring; Docker packages applications and dependencies so they run identically everywhere. **Adopted:** an Airflow DAG for the seven pipeline steps and a Docker Compose environment.

Observability and alerting

Production pipelines are monitored through run metadata (duration, status, volumes) and notified through alerts, so failures are found by the team rather than by users. Airflow [12] supports failure callbacks; Telegram [18] and email provide simple delivery channels. **Adopted:** a run-log table, a Pipeline Health page and alerts on task failure and large price moves.

Testing, CI and APIs

Automated tests (pytest [17]) run on every change by a CI service such as GitHub Actions [16] keep the main branch healthy; a REST API built with FastAPI [15] publishes interactive OpenAPI (Swagger) documentation automatically. **Adopted:** unit and idempotency tests in CI and an API for the latest data and detected changes.

2.4 Gaps Found and How We Address Them

Gap in existing solutions	Our response
Consumer trackers and SaaS tools hide their data pipeline.	A fully transparent, documented, open pipeline that can be inspected and extended.
Scraping tools stop at extraction.	Extraction is only step one; validation, deduplication, modelling and analysis follow.
Hobby projects lack reliability engineering.	Retries, logging, idempotent loads, quality gates and a recovery mode, tested on real failures.
Few solutions connect web data to a retailer's own catalogue.	Built-in comparison with an internal product catalogue.

2.5 Feedback, Evaluation and Suggested Improvements

Lecturer feedback and evaluation: to be completed by the Technical Instructor after reviewing this document.
Final grading criteria: as provided by the DEPI initiative (documentation, implementation, testing, presentation).

Improvements identified by the team for later phases:

- Add more than one data source (a second website or an API) to show that new sources only need a contract-compliant scraper module.
- Add a star-schema layer (fact_price, dim_product, dim_date) on top of the current tables.
- Add price anomaly detection (IQR or z-score) and flag unusual movements on the dashboard.
- Publish the dashboard online so the project can be demonstrated through a live link.

2.6 References

[1] Keepa, Amazon price tracker. https://keepa.com
[2] CamelCamelCamel, Amazon price tracker. https://camelcamelcamel.com
[3] Prisync, competitor price tracking and dynamic pricing software. https://prisync.com
[4] Scrapy Documentation. https://docs.scrapy.org
[5] Koster, M., Illyes, G., Zeller, H., Sassman, L. *Robots Exclusion Protocol*, RFC 9309, IETF, 2022. https://www.rfc-editor.org/rfc/rfc9309
[6] Richardson, L. *Beautiful Soup Documentation*. https://www.crummy.com/software/BeautifulSoup/bs4/doc/
[7] GitHub Topics: price-tracker (open-source price tracking projects). https://github.com/topics/price-tracker
[8] Kimball, R., Ross, M. *The Data Warehouse Toolkit*, 3rd ed., Wiley, 2013.
[9] Levenshtein, V. I. "Binary codes capable of correcting deletions, insertions and reversals", *Soviet Physics Doklady*, 10(8), 1966.
[10] RapidFuzz Documentation. https://rapidfuzz.github.io/RapidFuzz/
[11] Great Expectations Documentation. https://greatexpectations.io
[12] Apache Airflow Documentation. https://airflow.apache.org/docs/
[13] Books to Scrape (practice website). https://books.toscrape.com
[14] MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/
[15] FastAPI Documentation. https://fastapi.tiangolo.com
[16] GitHub Actions Documentation. https://docs.github.com/en/actions
[17] pytest Documentation. https://docs.pytest.org
[18] Telegram Bot API. https://core.telegram.org/bots/api

3. Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder	Role / interest	Needs from the system	Influence
Pricing / e-commerce manager	Sets and reviews prices	See how product prices change over time; spot products where the web price differs from the internal price.	High
Merchandising / category manager	Manages the assortment	Know which products are new, which disappeared and which changed category.	High
Data analyst / BI user	Builds reports	Clean, deduplicated, well-modelled data; ready-made SQL views; a dashboard.	Medium
Data engineer / platform owner	Operates the pipeline	Automated daily runs, retries, logs, quality results, easy deployment.	High
Source website owner (indirect)	Hosts the data	Polite, low-frequency access that respects robots.txt and terms.	Medium
DEPI instructors and evaluators	Assess the project	Clear documentation, working demo, evidence of testing and teamwork.	High
Project team (5 members)	Build the system	Clear interfaces, parallel work, a stable shared repository.	High

3.2 User Stories and Use Cases

3.2.1 User stories

ID	User story	Acceptance criteria
US-01	As a **pricing manager**, I want a daily snapshot of every product so that I can see how prices moved.	Each run adds one row per product with the snapshot date.
US-02	As a **pricing manager**, I want a list of products whose price changed since the previous snapshot.	Old price, new price, difference and percent change are shown.
US-03	As a **category manager**, I want to know which products are new and which were removed.	New and removed products are listed per run.
US-04	As a **category manager**, I want to be told when a product moves to another category.	Old and new category are shown with the date.
US-05	As a **pricing manager**, I want to compare web prices with our internal catalogue.	Matched products show the web price, internal price and gap.
US-06	As a **data analyst**, I want likely duplicates flagged for review, not silently deleted.	A report lists name pairs with a similarity score of 90% or more.
US-07	As a **data engineer**, I want the pipeline to run automatically every day and retry on failure.	Airflow shows a scheduled DAG, 3 retries and a failure callback.
US-08	As a **data engineer**, I want to re-run the pipeline safely.	A second run on the same data adds no rows.
US-09	As a **data steward**, I want to see rejected rows and failed quality checks.	Rejected rows are in a separate file; check results are in a log table.
US-10	As a **dashboard user**, I want charts instead of SQL.	Dashboard shows price distribution, price changes, new/removed per run, category changes.

ID	User story	Acceptance criteria
US-11	As a **developer**, I want to fetch the latest data through a documented REST API.	Endpoints return latest products, price changes, new and removed products as JSON; Swagger page is available.
US-12	As a **data engineer**, I want to be alerted when the pipeline fails or a price moves sharply.	A Telegram/email message is sent after the final retry fails, and when a price change exceeds the threshold.
US-13	As a **data engineer**, I want to see pipeline health at a glance.	A dashboard page shows recent runs with duration, status, rows loaded and rejected.
US-14	As a **team member**, I want tests to run automatically on every pull request.	A GitHub Actions workflow runs the tests and reports pass or fail on the pull request.

3.2.2 Use-case diagram

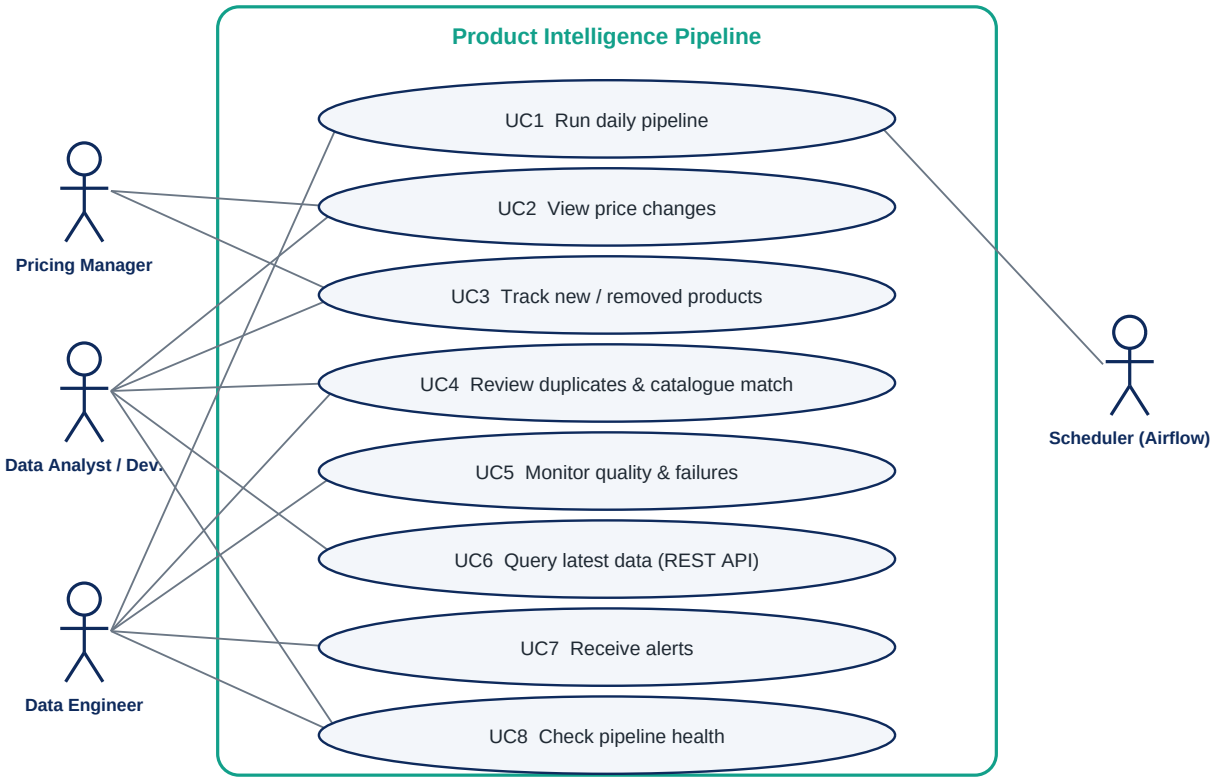


Figure 3. Actors and main use cases of the system.

3.2.3 Use-case descriptions

Use case	Actor	Pre-condition	Main flow	Result
UC1 Run daily pipeline	Scheduler, Data engineer	Environment running; source reachable	Scrape, validate, clean, deduplicate, load, run analysis, quality check	New snapshot stored; task logs written
UC2 View price changes	Pricing manager, Analyst	At least two snapshots exist	Open dashboard or query view; select period	Table/chart of changed prices
UC3 Track new and removed products	Pricing manager, Analyst	At least two snapshots exist	Open the new/removed view	Lists of new and removed products
UC4 Review duplicates and catalogue match	Data engineer, Analyst	A run has completed	Open the possible-duplicates and catalogue-match reports	Reviewed list; decisions recorded

Use case	Actor	Pre-condition	Main flow	Result
UC5 Monitor quality and failures	Data engineer	DAG has run	Check Airflow status and the quality log; inspect rejected rows	Issue identified and fixed or accepted
UC6 Query latest data (API)	Developer, Analyst	API running	Call /products/latest or /products/price-changes	JSON response
UC7 Receive alerts	Data engineer	Alert channel configured	A task fails after retries, or a price moves above the threshold; the system sends a message	Alert received with task or product details
UC8 Check pipeline health	Data engineer	At least one run logged	Open the Pipeline Health page	Run history with duration, status and row counts

3.3 Functional Requirements

Priority: **M** = must have, **S** = should have, **C** = could have (optional).

ID	Area	Requirement	Prio.
FR-01	Ingestion	Scrape the product list across all result pages (50 pages) and visit each product page.	M
FR-02	Ingestion	Extract name, category, price, rating, availability, product URL and scrape timestamp.	M
FR-03	Ingestion	Respect robots.txt; wait at least 1 second between requests; send an identifiable User-Agent.	M
FR-04	Ingestion	Retry failed requests up to 3 times; log every failure; one failed product must not stop the run.	M
FR-05	Ingestion	Save the raw snapshot as a dated JSON file following the Data Contract.	M
FR-06	Ingestion	Provide a mode that re-collects only the products that failed earlier.	S
FR-07	Cleaning	Validate that all required columns are present and not empty.	M
FR-08	Cleaning	Trim and normalise product names and category names.	M
FR-09	Cleaning	Convert price text to a numeric value and a separate currency code.	M
FR-10	Cleaning	Convert rating words (One to Five) to integers 1 to 5.	M
FR-11	Cleaning	Standardise availability to two values: In stock / Out of stock.	M
FR-12	Cleaning	Apply missing-value rules: reject rows without URL, name or price; set a missing category to Unknown.	M
FR-13	Cleaning	Write rejected rows to a separate file instead of discarding them.	M
FR-14	Duplicates	Remove exact duplicates using product URL as the unique key.	M
FR-15	Duplicates	Flag near-duplicate product names (similarity of about 90% or more) in a review report.	M
FR-16	Duplicates	Compare scraped products with the internal catalogue and report matched and unmatched items.	M
FR-17	Storage	Load data into a staging table, then upsert into the products table.	M
FR-18	Storage	Append one snapshot row per product per day to price_history.	M
FR-19	Storage	Re-running the load for the same day must not create duplicate rows.	M
FR-20	Storage	Mark products missing from the latest snapshot as inactive.	M
FR-21	Analysis	Provide SQL views for price changes, new products, removed products and category changes.	M

ID	Area	Requirement	Prio.
FR-22	Analysis	Provide a category-statistics view (average price and product count per category).	S
FR-23	Orchestration	Provide an Airflow DAG: scrape, validate, clean, dedup, load, run analysis, quality check.	M
FR-24	Orchestration	Run daily with 3 retries and a failure callback; log every task.	M
FR-25	Quality	Run quality checks (row count, nulls, negative prices, rating range, duplicates, rejected ratio).	M
FR-26	Quality	Store check results in a log table and file; a failed check must fail the pipeline task.	M
FR-27	Presentation	Provide a dashboard with at least three visualisations built from the SQL views.	M
FR-28	API	Expose /products/latest, /products/price-changes, /products/new and /products/removed through a REST API with automatic Swagger documentation.	M
FR-29	Operations	Start the full environment (Airflow, MySQL) with one Docker Compose command.	M
FR-30	Operations	Keep credentials (database, alert tokens) in a .env file or GitHub Secrets, never in version control.	M
FR-31	Alerts	Send a Telegram or email alert when a DAG task fails after its retries.	S
FR-32	Alerts	Send an alert when a product price changes by more than a configurable threshold (default 20%).	S
FR-33	Observability	Record every pipeline run in a pipeline_run_log table: start, end, duration, status, rows scraped, loaded and rejected.	S
FR-34	Observability	Provide a Pipeline Health dashboard page that shows the latest runs and trends from the run-log table.	S
FR-35	Testing	Provide pytest unit tests for the cleaning and deduplication functions and an idempotency test.	S
FR-36	Testing	Run the tests automatically on every pull request with a GitHub Actions workflow.	S

3.4 Non-Functional Requirements

ID	Category	Requirement	Measurable criterion
NFR-01	Performance	Polite scraping must still finish in a reasonable time.	Full 1,000-product scrape within about 60 minutes; clean + dedup + load under 2 minutes; views answer in under 2 seconds.
NFR-02	Reliability	Transient failures must not stop or corrupt a run.	3 retries per request and per Airflow task; at least 95% of scheduled runs succeed.
NFR-03	Data integrity	Loads are idempotent and consistent.	UNIQUE (product_id, snapshot_date); single transaction; 0 extra rows after a re-run.
NFR-04	Data quality	Only valid data reaches the warehouse.	Rejected rows under 5%; 0 negative prices; ratings in 1 to 5; 0 duplicate URLs.
NFR-05	Security	Protect credentials and the repository.	No secrets in Git; .env ignored; protected main branch; changes via pull request.
NFR-06	Usability	Non-technical users can read the results.	Dashboard needs no SQL; README lets a new user run the project from a clean clone.
NFR-07	Maintainability	Code is modular and documented.	Separate modules per step; documented functions; agreed naming conventions; code review through pull requests.

ID	Category	Requirement	Measurable criterion
NFR-08	Portability	Runs the same on any team laptop.	Docker Compose environment; works on Windows and Linux.
NFR-09	Scalability	New sources and larger volumes can be added.	A new source only needs a scraper producing Data-Contract JSON; indexes on lookup columns.
NFR-10	Observability	Operators can see what happened.	Log file per component; Airflow task logs; quality-log table.
NFR-11	Legal / ethical	Collect data responsibly.	robots.txt checked and documented; 1 s delay; no personal data; no login-protected content.
NFR-12	Testability	Key behaviour is verified.	pytest unit tests for cleaning and deduplication; idempotency test; seeded change-detection test; all run in CI.
NFR-13	API performance	The API answers quickly and is documented.	/products/latest returns 50 items in under 500 ms; Swagger documentation generated automatically.
NFR-14	CI efficiency	CI gives fast feedback.	The GitHub Actions workflow finishes in under 5 minutes on a pull request.
NFR-15	Alert timeliness	Failures are reported promptly.	Failure alert sent within 2 minutes of the final failed retry; no alert flooding (one per failed task per run).

3.5 Assumptions and Constraints

- **Single source:** books.toscrape.com stands in for a retailer's public web source; its data is demonstration data (random prices and ratings).
- **Technology constraints:** Python, SQL (MySQL 8), Apache Airflow and Docker, as required by the DEPI Data Engineering track.
- **Time:** deliverables follow the DEPI documentation deadlines (see section 1.2.3).
- **Team:** five members working in parallel; interfaces are fixed by the Data Contract and Handoff Interface.
- **Infrastructure:** local laptops and free tools only (including free tiers of GitHub Actions and Telegram bots); no cloud costs.
- **Assumption:** the source website stays available and keeps a similar HTML structure during development.

3.6 Traceability: Requirements to Team Members

Member	Primary requirements covered
Member 1 (Marwa Yosry)	FR-01 to FR-06, FR-27, FR-34, NFR-11 (and Data Contract / repository governance)
Member 2 (Gany Mohamed)	FR-17 to FR-20 (schema side), FR-21, FR-22, FR-33 (run-log table), NFR-03, NFR-09
Member 3 (Ayat Abdelaziz)	FR-07 to FR-13, FR-25, FR-26, FR-35, NFR-04, NFR-12
Member 4 (Carol Kitchener)	FR-14 to FR-20, FR-28 (REST API), NFR-03, NFR-13
Member 5 (Engy Ashraf) with Team Leader	FR-23, FR-24, FR-29, FR-30, FR-31, FR-32, FR-33 (writing run records), FR-36, NFR-02, NFR-05, NFR-08, NFR-10, NFR-14, NFR-15; integration testing

Next step

Part 2 of the documentation (System Analysis & Design: ERD, schemas, DFD, sequence/activity/state/class diagrams, UI mock-ups, deployment and component diagrams) will build directly on the requirements above.